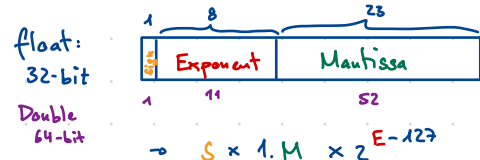


Number Representation

→ 32 bit Integer: $\in \{-2^{31}, 2^{31}-1\}$
 ↳ 0000 ... 0001 → +1
 ↳ 1000 ... 0001 → -1

→ Floatingpoint Numbers are harder

↳ IEEE-754 Standard:



Rounding

→ In CS Round to nearest Even is standard

1.75 → 1.8 / 1.65 → 1.6

Root Finding (Nullpunkte)

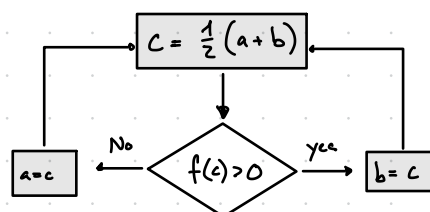
→ Midnight Formula in CS:

$$q = -\frac{1}{2} (b + \text{sign}(b) \sqrt{b^2 - 4ac})$$

$$x_1 = \frac{q}{a} \quad x_2 = \frac{c}{q}$$

→ There are also Formulas for cubic and quartic but no further

→ Algo:

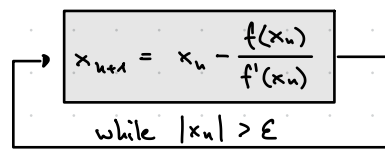
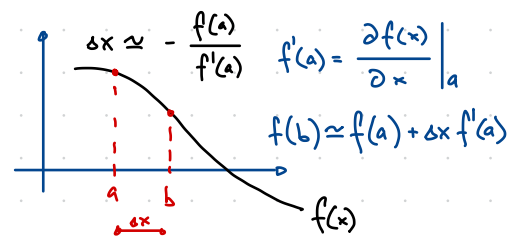


→ until $|a-b| < \epsilon_{\text{Absolute}}$ or $\frac{|a-b|}{|c|} < \epsilon_{\text{Relative}}$

→ Until a precise float → 23 Iterations
 → Until a precise double → 52 Iterations

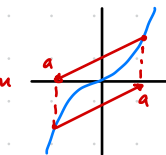
↳ Linearly convergent
 ↳ Constant Number of Digits per Iteration
 ↳ Expensive

Root Finding - Newton's Method



→ Does not always work!

e.g. for cubic function it gets cyclic



Population Growth

Unlimited Growth:

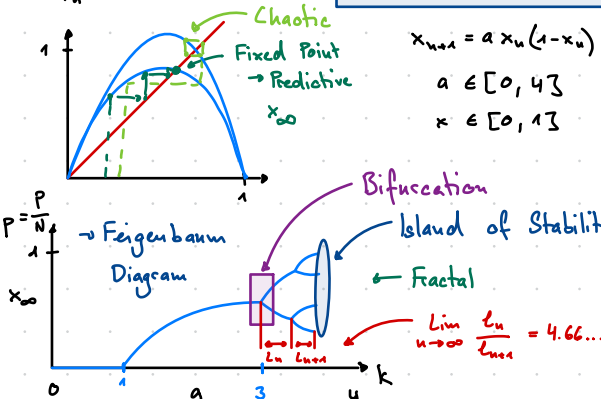
$$P_{n+1} = (1+r)P_n$$

$$P_n(P_0) = (1+r)^n P_0$$

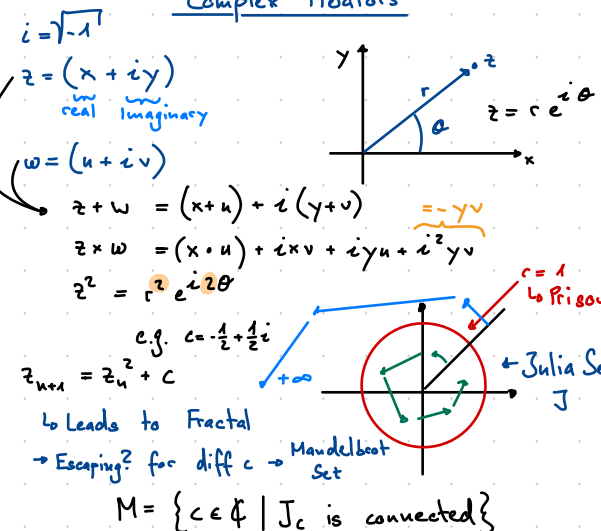
$$r = \frac{P_{n+1} - P_n}{P_n}$$

For a more realistic Calculation, we introduce N
 ↳ N = sustainable Size of Population
 ↳ r now corresponds to (1-p) $p = \frac{P}{N}$
 ↳ Verhulst Model: $r = k(1-p)$

$$\frac{P_{n+1} - P_n}{P_n} = k(1-p) \rightarrow P_{n+1} = P_n + P_n k(1-p)$$

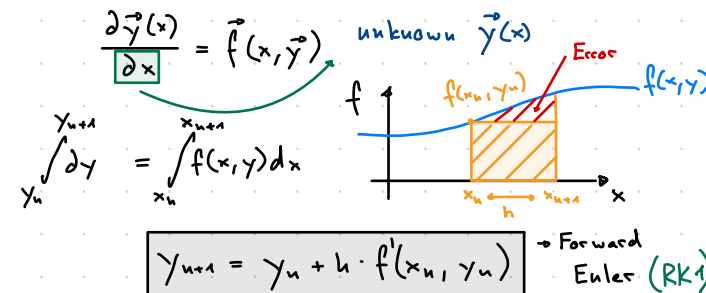


Complex Iterators



Simulations in Science

Ordinary Differential Equations



Types of Error

- ① Truncation: Error associated with Method
- ② Round-off: Error through precision of float

Error of Forward Euler:

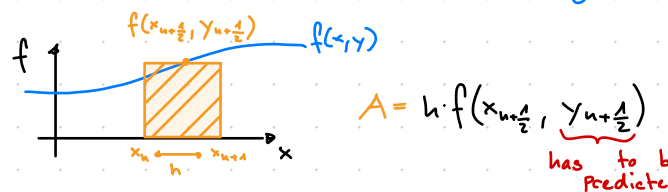
$$\int_{x_n}^{x_{n+1}} f(x, y(x)) dx \approx \int_{x_n}^{x_{n+1}} (f(x_n, y_n) + h \cdot f'(x_n, y_n)) dx$$

$$y_{n+1} - y_n \approx h \cdot f(x_n, y_n) + \frac{h^2}{2} f''(x_n, y_n)$$

Error of Single Step → $O(h^2)$

$$\text{Global Error: } N_{\text{steps}} \times O(h^2) = O(h^2)$$

↳ Midpoint Method is better = Runge Kutta II



Predictor: $y_{n+1/2}^* = y_n + \frac{h}{2} f(x_n, y_n)$ (RK2)
 Corrector: $y_{n+1}^* = y_n + h f(x_{n+1/2}, y_{n+1/2}^*)$

```
def forward_euler(x0, y0, h, steps):
    x_vals, y_vals = [x0], [y0]
    x, y = x0, y0
    while steps > 0:
        dxdt, dydt = lotka_voltterra(x, y)
        x += h * dxdt
        y += h * dydt
        x_vals.append(x)
        y_vals.append(y)
        steps -= 1
    return np.array(x_vals), np.array(y_vals)
```

```
def midpoint_method(x0, y0, h, steps):
    x_vals, y_vals = [x0], [y0]
    x, y = x0, y0
    while steps > 0:
        dxdt1, dydt1 = lotka_voltterra(x, y)
        x_mid = x + h/2 * dxdt1
        y_mid = y + h/2 * dydt1
        dxdt2, dydt2 = lotka_voltterra(x_mid, y_mid)
        x += h * dxdt2
        y += h * dydt2
        x_vals.append(x)
        y_vals.append(y)
        steps -= 1
    return np.array(x_vals), np.array(y_vals)
```

"Black-Box Methods"
 → good if we have little knowledge about problem

Lotka Volterra: $\frac{df}{dt} = -k_f \cdot f + k_{fm} \cdot f \cdot m$ | foxes
 $\frac{dm}{dt} = -k_m \cdot m - k_{mf} \cdot m \cdot f$ | mice

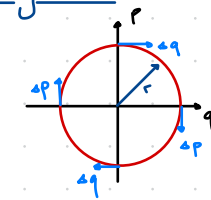
Symplectic Integrators

Example Harmonic Oscillator:

$$\dot{p} = -q$$

$$\dot{q} = p$$

Hamiltonian Equations

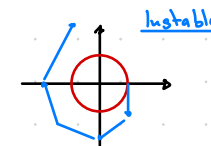


→ Conserved Energy Phase Space

Forward Euler:

$$q_{n+1} = q_n + h p_n$$

$$p_{n+1} = p_n - h q_n$$

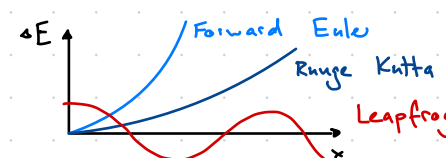
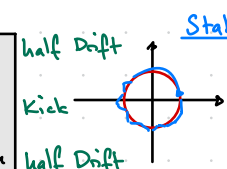


Leap Frog:

$$q_{n+1/2} = q_n + \frac{1}{2} h p_n$$

$$p_{n+1} = p_n - h q_{n+1/2}$$

$$q_{n+1} = q_{n+1/2} + \frac{1}{2} h p_{n+1}$$



N-Body Problem

Hamiltonian: Function of all Energy in System

$$H = T + U$$

T = Kinetic Energy
 U = Potential Energy

→ The amount of different Forces in a System with N Bodies: $\frac{N(N-1)}{2}$
 ↳ Newton's 3rd: $F_{ij} = -F_{ji}$

→ Math Tricks can get the complexity from $O(N^2)$ to $O(N \log N)$ to $O(N)$

$$\vec{F}_{ij} = \frac{G m_i m_j}{|\vec{r}_i - \vec{r}_j|^2} \cdot \frac{(\vec{r}_j - \vec{r}_i)}{|\vec{r}_j - \vec{r}_i|}$$

$$\vec{F}_i = \sum_{j \neq i} \frac{G m_i m_j}{|\vec{r}_i - \vec{r}_j|^2} (\vec{r}_j - \vec{r}_i)$$

→ Link between Hamiltonian Framework and Newtonian Mechanics:

$$\dot{\vec{p}} = -\frac{\partial H}{\partial \vec{q}} = -\frac{\partial U(\vec{q})}{\partial \vec{q}} = -\nabla U = \frac{\vec{F}_i}{m_i} = \vec{a}_i$$

→ We Can use Leapfrog: h = Time Step

$$\vec{r}_{i,n+1/2} = \vec{r}_{i,n} + \frac{h}{2} \vec{v}_{i,n}, \forall i$$

$$\vec{v}_{i,n+1} = \vec{v}_{i,n} + h \vec{a}_i(\vec{r}_{i,n+1/2}), \forall i$$

$$\vec{r}_{i,n+1} = \vec{r}_{i,n+1/2} + \frac{h}{2} \vec{v}_{i,n+1}, \forall i$$

Parabolic P.D.E's

→ Describe Phenomena of "Diffusing"
 ↳ Heat Equation for Example

→ Parabolic PDE's rely on initial/boundary conditions

→ Create a Grid with Sidelength J & L

↳ grid spacings Δx & Δy
 ↳ $x_j = x_0 + j \Delta x, j \in [0, J]$
 ↳ $y_L = y_0 + L \Delta y, L \in [0, L]$

→ Stencil describes how each Grid point depends on surrounding Gridpoints

$$\forall_{j,L} \frac{1}{h^2} (\phi_{j+1,L} + \phi_{j-1,L} + \phi_{j,L+1} + \phi_{j,L-1} - 4\phi_{j,L}) = 0$$

→ This Stencil holds for all Laplacians

Jacobi's Method

$$\phi_{j,L}^{\text{new}} = \frac{1}{4} [\phi_{j+1,L}^{\text{old}} + \phi_{j-1,L}^{\text{old}} + \phi_{j,L+1}^{\text{old}} + \phi_{j,L-1}^{\text{old}} - 4\phi_{j,L}^{\text{old}}] = 0$$

$O(N^4)$

Successive Over Relaxation

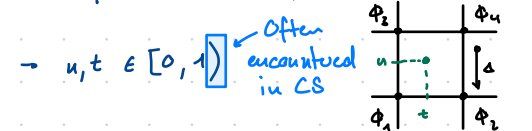
$$\phi_{j,L}^{\text{new}} = \phi_{j,L}^{\text{old}} + \omega [\phi_{j+1,L}^{\text{old}} + \phi_{j-1,L}^{\text{old}} + \phi_{j,L+1}^{\text{old}} + \phi_{j,L-1}^{\text{old}} - 4\phi_{j,L}^{\text{old}}]$$

$1 \leq \omega \leq 2$

→ With a checkbox the algorithm is sped up further

Interpolation

→ To estimate position which do not represent a Gridpoint



$$\phi(x,y) = (1-s)(1-t)\phi_1 + (1-s)t\phi_2 + s(1-t)\phi_3 + st\phi_4$$

$$\dot{x} = \left(\frac{e}{m_e}\right) \left(-\frac{1}{2} ((1-s)(\phi_2 - \phi_1) + s(\phi_4 - \phi_3))\right)$$

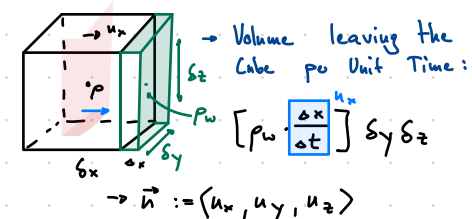
$$\dot{y} = \left(\frac{e}{m_e}\right) \left(-\frac{1}{2} ((1-t)(\phi_3 - \phi_1) + t(\phi_4 - \phi_2))\right)$$

→ Solvable with Leapfrog or RK4



Hyperbolic P.D.E.'s

→ Movement of Conserved Quantity in Space



Leaving Cube: $\left\{ \rho u_x + \frac{1}{2} \Delta x \frac{\partial(\rho u_x)}{\partial x} \right\} \Delta y \Delta z$

→ ρu_x : Mass Flux in x-Direction

→ $x + \frac{1}{2} \Delta x$: Position of Righthand Plane

→ $\frac{\partial(\rho u_x)}{\partial x}$: Flux Changes in respect to spatial coordinate x

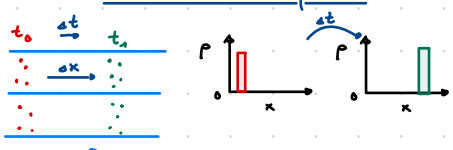
Entering Cube: $\left\{ \rho u_x - \frac{1}{2} \Delta x \frac{\partial(\rho u_x)}{\partial x} \right\} \Delta y \Delta z$

Conservation of Mass: $\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \vec{u}) = 0$

→ Divergence Operator

↳ If flow is steady: $\nabla \cdot (\rho \vec{u}) = 0$

Linear Advection Equation



First Method:
$$\rho_j^{(n+1)} = \rho_j^n - \frac{\Delta t}{\Delta x} c (\rho_{j+1}^n - \rho_{j-1}^n)$$

$$c = \frac{\Delta t u}{\Delta x}$$

→ Exercise Von Neuman Stability Analysis:

↳ To test whether errors grow/decrease over time

① Assume the solution can be written in Fourier Mode $\rho_j^n = \hat{\rho} e^{ikx}$

② Substitute F-Mode into numerical Scheme

③ Derive Amplification Factor A, which describes how amplitude evolves from one Timestep to the next

④ If $A > 1 \rightarrow$ unstable/stable else

$$A e^{ik\Delta x} = A e^{ik\Delta x} - \frac{\Delta t}{2} c (A e^{ik(j+1)\Delta x} - A e^{ik(j-1)\Delta x})$$

→ unstable? $A = 1 - c \left(\frac{e^{ik\Delta x} - e^{-ik\Delta x}}{2} \right)$
 $A = 1 - ic \sin \theta > 1 \forall \theta$

LAX Method:

$$\rho_j^{n+1} = \frac{1}{2} (\rho_{j+1}^n + \rho_{j-1}^n) - \frac{\Delta t}{2} c (\rho_{j+1}^n - \rho_{j-1}^n)$$

↳ Stable if $\frac{\Delta t}{\Delta x} u < 1$

Upwind Method:

$$\rho_j^{n+1} = \begin{cases} \rho_j^n - c (\rho_j^n - \rho_{j-1}^n) & , u \geq 0 \\ \rho_j^n - c (\rho_{j+1}^n - \rho_j^n) & , u < 0 \end{cases}$$

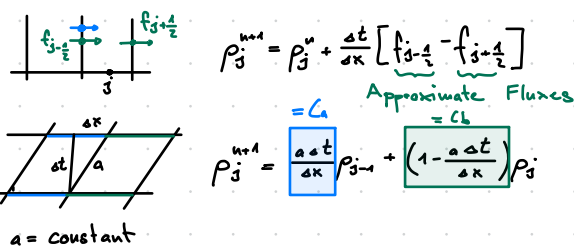
LAX - Wendroff Method

$$\rho_j^{n+1} = \frac{1}{2} c (1+c) \rho_{j-1}^n + (1-c^2) \rho_j^n - \frac{1}{2} c (1-c) \rho_{j+1}^n$$

$\hookrightarrow u > 0$

Finite Volume

→ Derivative functions can fail to describe conservation laws



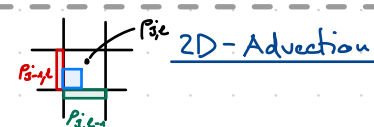
→ Laxo Method:

$$\rho_j^{n+1} = \rho_j^n - \frac{\Delta t}{2 \Delta x} (\rho_{j+1}^n - \rho_{j-1}^n)$$

→ Unstable

→ CIR-Method: (=1st Order Upwind)

$$\rho_j^{n+1} = \rho_j^n - \frac{\Delta t}{\Delta x} (\rho_j^n - \rho_{j-1}^n)$$



$$\rho_{j,e}^{n+1} = \rho_{j,e}^n - \frac{\Delta t}{\Delta x} (c_a (\rho_{j,e}^n - \rho_{j-1,e}^n) - c_b (\rho_{j,e}^n - \rho_{j,e-1}^n))$$

↳ Stable if $\frac{\Delta t}{\Delta x} + \frac{b \Delta t}{\Delta y} \leq 1$

→ Coruo Transport Upwind Scheme (CTU)

$$\rho_{j,e}^{n+1} = (1-c_a)(1-c_b) \rho_{j,e}^n + c_a(1-c_b) \rho_{j-1,e}^n + (1-c_a)c_b \rho_{j,e-1}^n + c_a c_b \rho_{j-1,e-1}^n$$

$c_a, c_b \in [0, 1]$

1D Hydrodynamics

→ Rewrite Methods using Fluxes: $c = \frac{\Delta t u}{\Delta x}$, $f(\rho) = \rho u$

LAX Method:

$$\rho_j^{n+1} = \frac{1}{2} (\rho_{j-1}^n + \rho_{j+1}^n) - \frac{\Delta t}{2 \Delta x} (F_{j+1}^n - F_{j-1}^n)$$

1st Order Upwind:

$$\rho_j^{n+1} = \begin{cases} \rho_j^n + \frac{\Delta t}{\Delta x} (F_j^n - F_{j-1}^n) & , u \geq 0 \\ \rho_j^n + \frac{\Delta t}{\Delta x} (F_{j+1}^n - F_j^n) & , u < 0 \end{cases}$$

→ Conservation of 3 Quantities:

① Cons. of Mass: $F = \rho u$

② Cons. of Momentum: $F = \rho u^2 + P$

③ Cons. of Energy: $F = u(E, P)$

Does not behave like Vektor $\vec{u} = \begin{pmatrix} \rho \\ \rho u \\ E \end{pmatrix}$ $F(u) = [\rho u, \rho u^2 + P, u(E+P)]$

↳ 4 Unknowns! $E = \text{kin. Energy} + \text{Thermal Energy}$
 $\frac{1}{2} \rho u^2$, $P = c \rho u^2$, $c = \text{thermal energy dens.}$

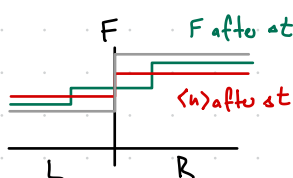
$$F(u) = [\rho u, \rho u^2 + P, u(\frac{1}{2} \rho u^2 + \frac{3}{2} P)]$$

→ u represents average value of Material in the Cell:

$$\langle u \rangle_i^n = \frac{1}{\Delta x} \int_{x_{i-\frac{1}{2}}}^{x_{i+\frac{1}{2}}} u(x, t^n) dx$$

→ This should be considered over a timestep

$$\frac{\langle u \rangle_i^{n+1} - \langle u \rangle_i^n}{\Delta t} + \frac{F_{i+\frac{1}{2}}^n - F_{i-\frac{1}{2}}^n}{\Delta x} = 0$$



$$D_L = u - c_s$$

$$D_R = u + c_s$$

$$D_{\max} = \max(D_L, D_R) = \sqrt{\gamma \frac{P}{\rho}}$$

$c_s = \text{Speed of Sound}$

LAX - Friedrichs Riemann Solver

$$F_{i-\frac{1}{2}}^n = \frac{1}{2} [F_i^n + F_{i-1}^n] - \frac{1}{2} D_{\max} [u_i^n - u_{i-1}^n]$$

↳ Provides the Flux at the Interface of two states

Finite Volume Method

$$\langle u \rangle_i^{n+1} = \langle u \rangle_i^n - \frac{\Delta t}{\Delta x} [F_{i+\frac{1}{2}}^n - F_{i-\frac{1}{2}}^n]$$

↳ Calculates Netflux across Boundary / updates Cell

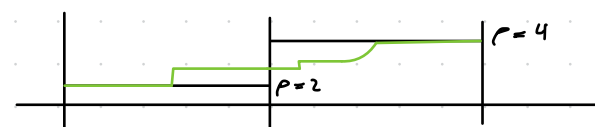
Finite Volume Method II

$$^* u_i^{n+\frac{1}{2}} = ^* u_i^n - \frac{\Delta t}{2 \Delta x} [F_{i+\frac{1}{2}}^n - F_{i-\frac{1}{2}}^n] \rightarrow \text{Predictor}$$

$$u_i^{n+1} = u_i^n - \frac{\Delta t}{\Delta x} [F(^* u_i, ^* u_{i+1}) - F(^* u_{i-1}, ^* u_i)] \rightarrow \text{Corrector}$$

Friedrichs Riemann Solver

SOD - Shock Tube Problem



P.D.E.'s!

$$A \frac{\partial^2 u}{\partial x^2} + B \frac{\partial^2 u}{\partial x \partial y} + C \frac{\partial^2 u}{\partial y^2} = 0$$

$B^2 - 4AC < 0 \rightarrow$ Elliptic P.D.E.

↳ Equilibrium State (Laplace)

↳ Jacobi, SOR

$$\phi_{j,e}^{new} = \phi_{j,e}^{old} + \frac{\omega}{4} \left[\phi_{j-1}^{old} + \phi_{j+1}^{old} + \phi_{j,e-1}^{old} + \phi_{j,e+1}^{old} \right]$$

$1 \leq \omega \leq 2$

$B^2 - 4AC = 0 \rightarrow$ Parabolic P.D.E.

↳ Something Diffusing (Heat equation)

↳ Forward Eulo

$$u_i^{n+1} = u_i^n + \Delta t D \frac{u_{i-1}^n - 2u_i^n + u_{i+1}^n}{\Delta x^2}$$

$B^2 - 4AC > 0 \rightarrow$ Hyperbolic P.D.E.

↳ WaveLike Behaviors (Hydrodynamics)

↳ LAX, FirstOrderUpwind, CTU